

TurboGator

Adam Suma

Frederico Aberle

Jannis Alsbach

Mihai-George Licu



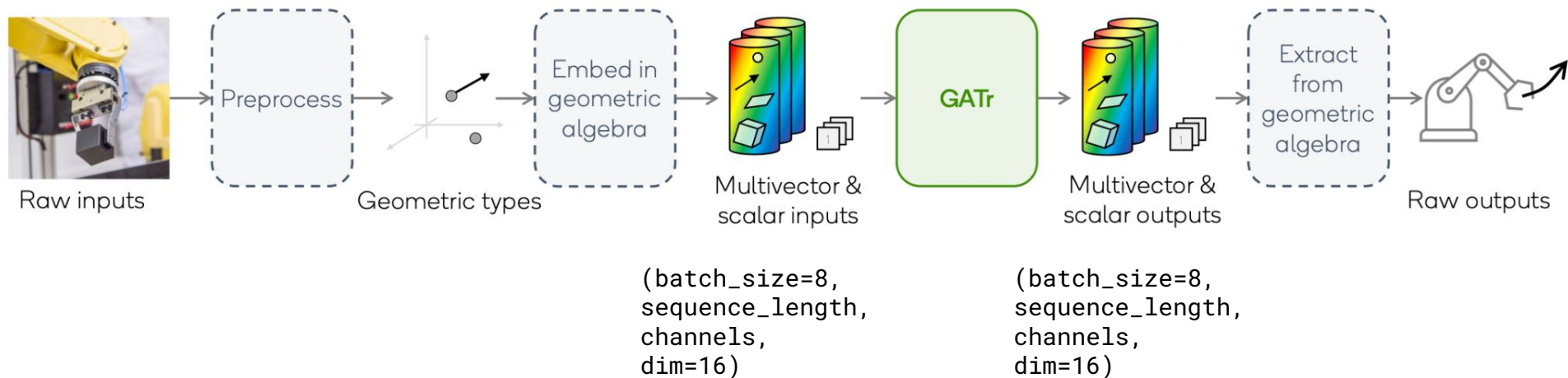
ETH

Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

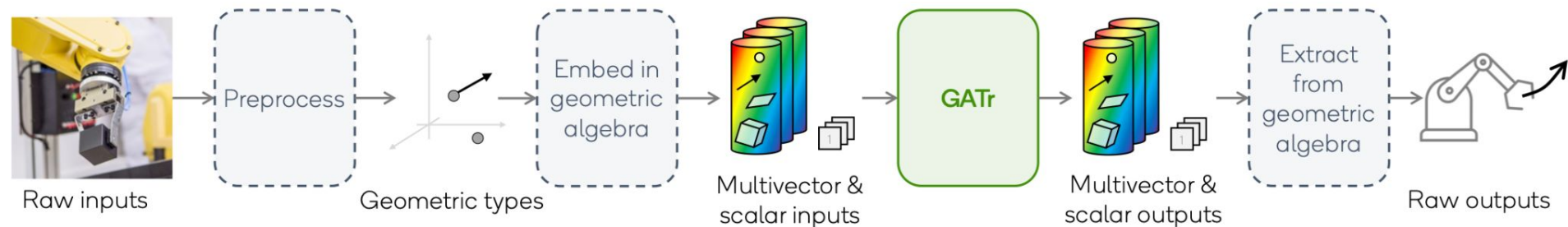
Todo Slides:

- Bottlenecks fully identified
- Bottlenecks optimized
 - Sparsity
 - Simplicity
 - Memory Layout Optimizations
 - Flattening & Contiguous vectors
 - Blocking ?
 - Others ?
- Memory layout optimization
- Runtime & roofline plots
- (BOLD) TODO Vectorization

The Core Algorithm

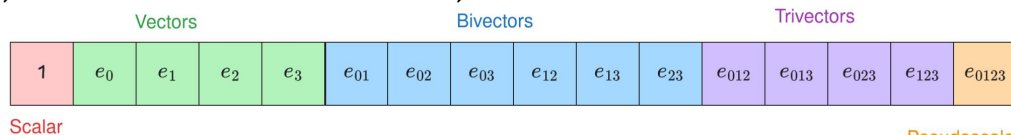


The Core Algorithm



(batch_size=8,
sequence_length,
channels,
dim=16)

(batch_size=8,
sequence_length,
channels,
dim=16)

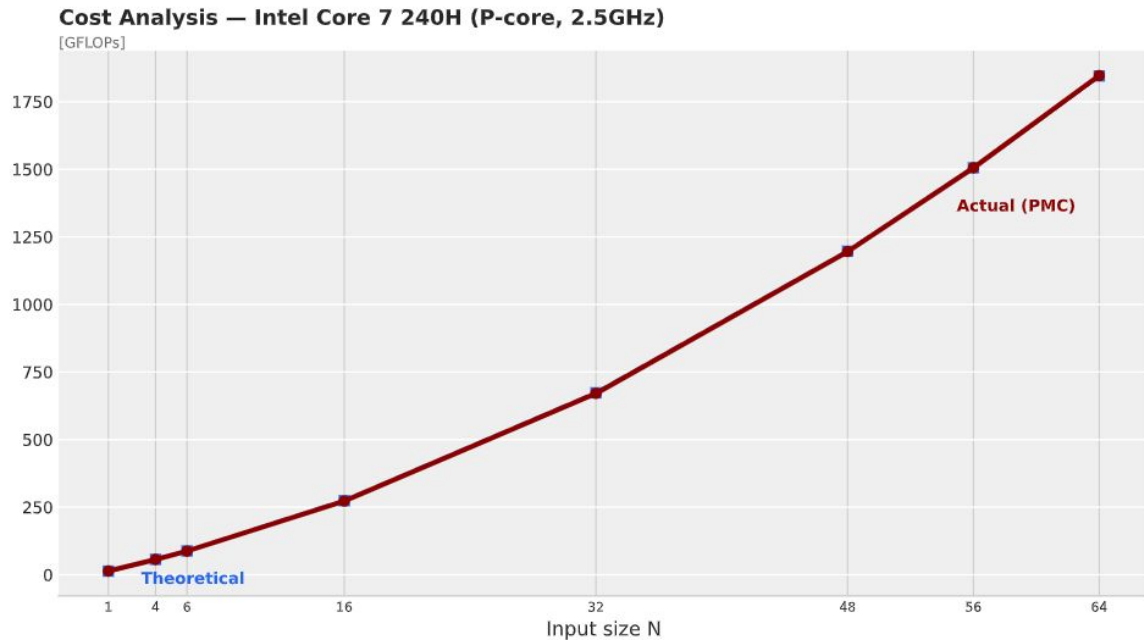


The Bottleneck: Dense Math on Sparse Data

- **Basis creates memory and computational overhead**
 - $16 \times 16 \times 16 = 4096$ floats allocated (mostly zero)
 - Multi-level looping
- **Introduction of new sparse data structure necessary**
 - Store non-zero coordinates with sign flips (~ 200 nnz)
 - Hardcode the output assignments

→ **Reduction of $\sim 95\%$ in data size**

Cost Analysis



Work (FLOPs) $\leq O(N^2)$

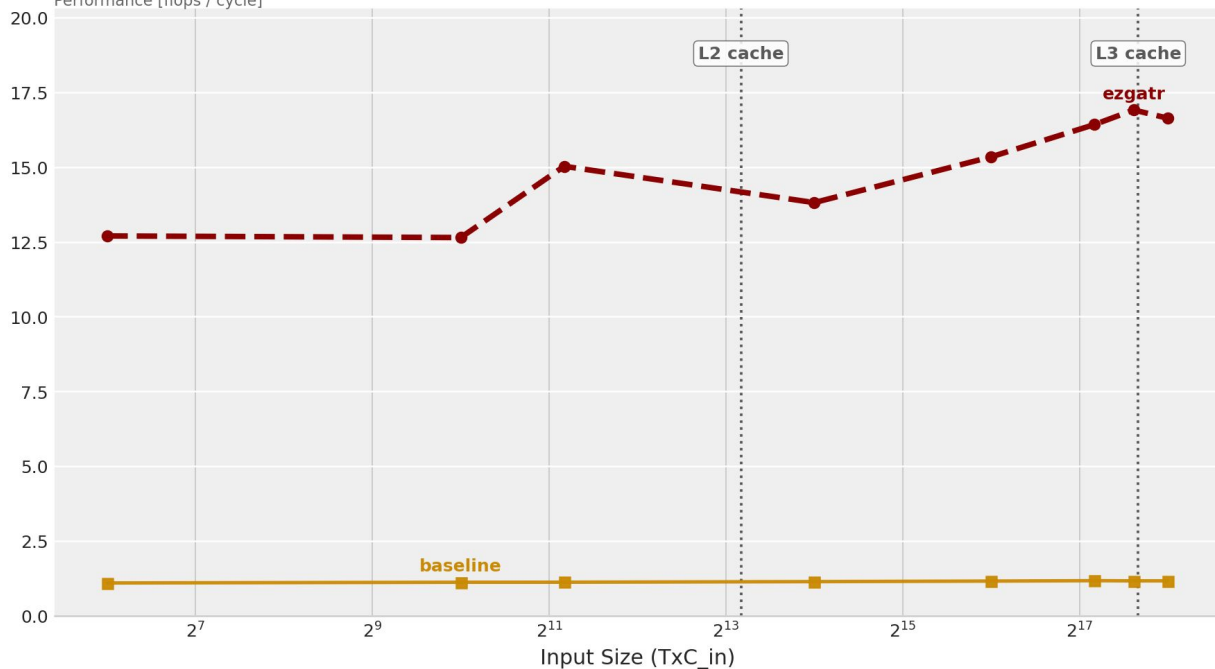
$$T = 32 * N$$

$$C_{in} = 2 * N$$

The naive C++ Baseline

Performance: Intel Core 7 240H (P-core, 2.5GHz)

Performance [flops / cycle]



2% of Theoretical Peak
(~0.8 / 40 FLOPs/cycle)

Optimized Versions

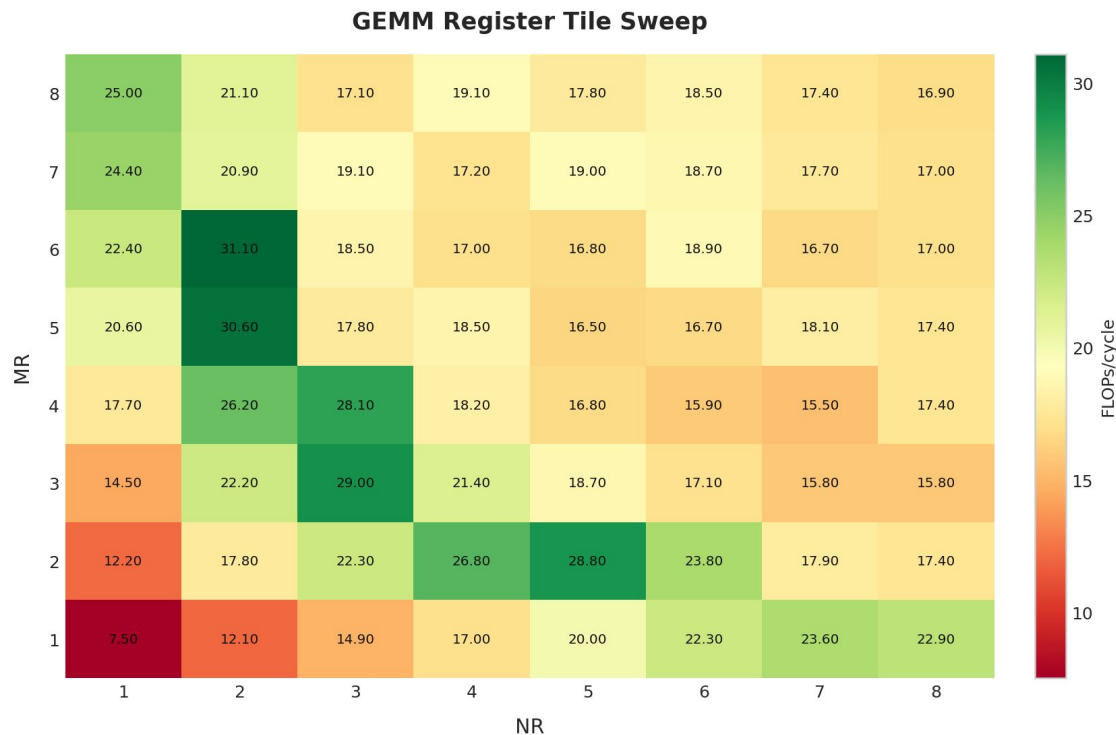
Incremental Improvements

- **Baseline**
 - Dense basis tensors, naive nested loops, basic contractions for equi_linear
- **ScalarGatorV1**
 - Sparsity + Simplicity + Cache Blocking + Scalar replacement
- **ScalarGatorV2 (best scalar version)**
 - Online Softmax + Flash Attention
- **TurboGator**
 - 256-bit AVX intrinsics with full pipelining
 - Glue layer improvements

Online Softmax & Flash Attention

- Classical (safe) Softmax:
 - 3 passes (find maximum, calculate exp+sum, normalize)
 - Online softmax:
 - 2 streaming passes with running (max, sum) + on-the-fly correction
 - Flash Attention (enabled via online Softmax)
 - Tile-wise fused $QK^T \rightarrow \text{softmax} \rightarrow V \cdot O$ with running (max, sum) correction across tiles
- $O(T \cdot d)$ memory traffic instead of $O(T^2)$

Optimal Tiling

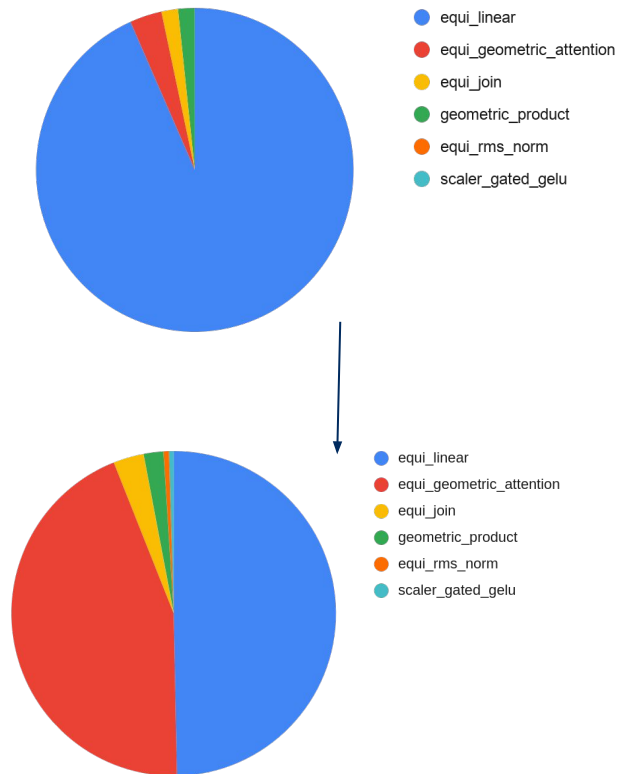


Best FLOPs/cycle performance: NR = 2, MR = 6 (31.10)

- Extracted optimum tiles for GEMM on our machine using a different simple matrix multiply program.
- For our specific task -> the optimal value was **4x2** because of divisibility to 32.

Profiling

Distribution of Trace Time by Function



ScalarGatorV1

equi_linear
149610ms

geom_attn
5226ms

160x

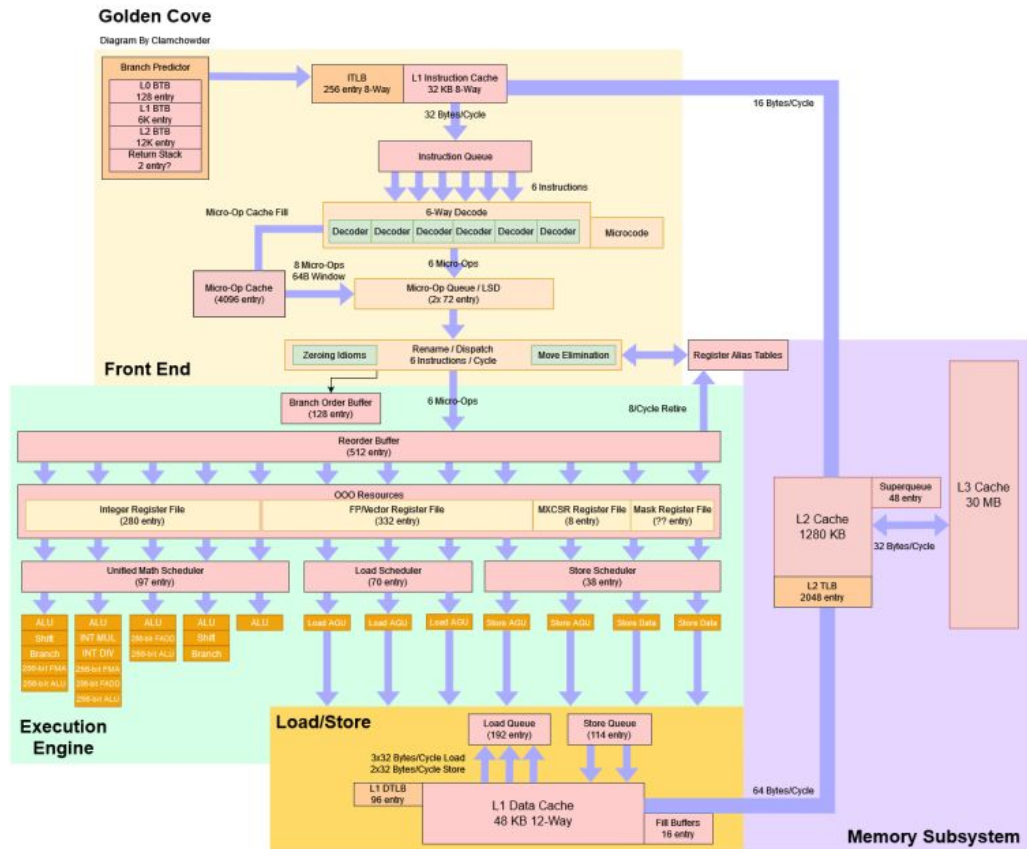
6.3x

TurboGator

932ms

831ms

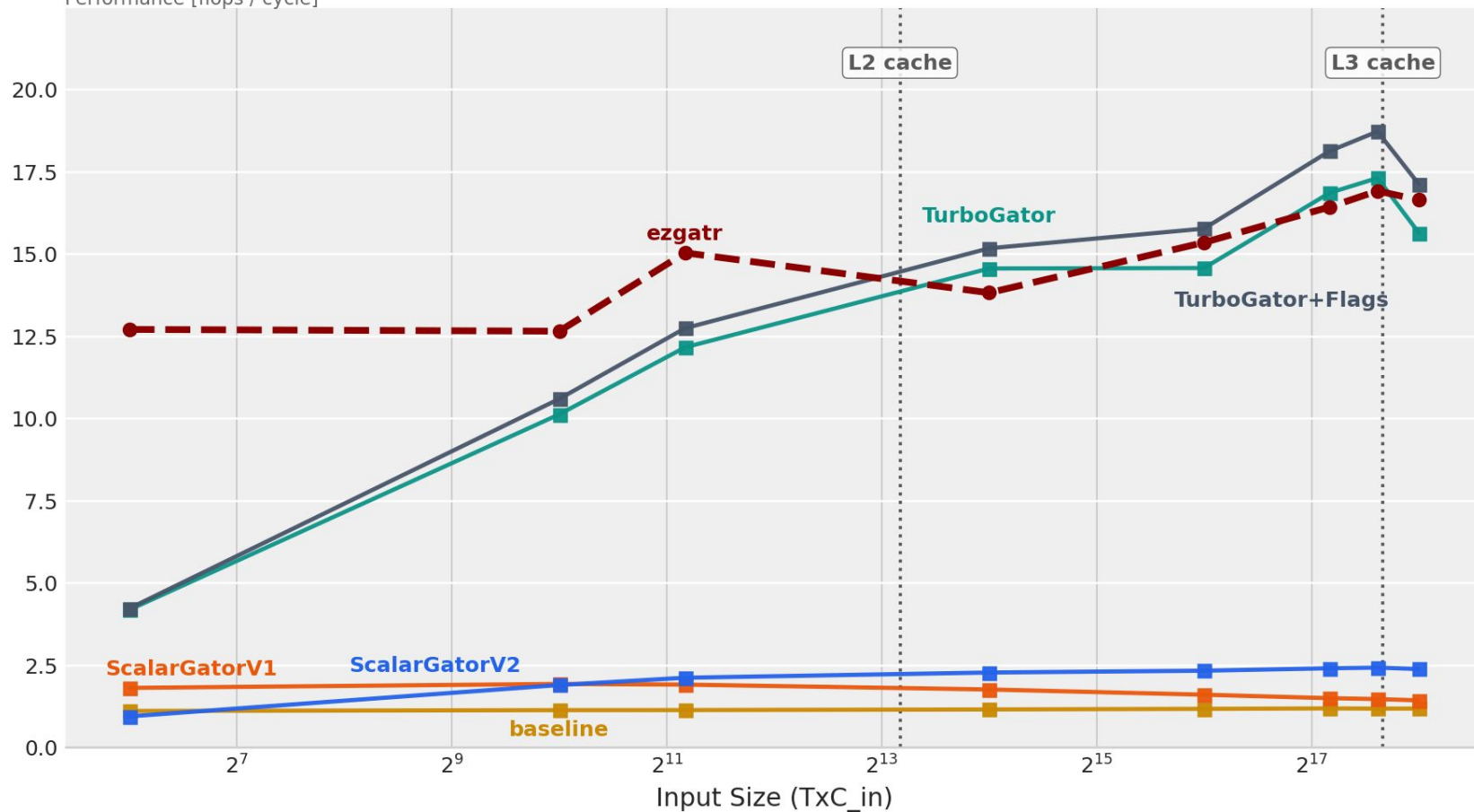
Server Setup



- Intel Core 7 240H Raptor Lake P-core (Golden Cove)
- Intel SSE4.1, SSE4.2, AVX2, FMA3
- TurboBoost off, pinned P-core 4 @ 2.5 GHz, 32GB LPDDR5
- $\pi_s = 5$ flops/cycle
- $\pi_v = 40$ flops/cycle
- β (L1) = 96 bytes/cycle
- β (L2) = 64 bytes/cycle
- β (L3) = 32 bytes/cycle
- β (DRAM) = 9 bytes/cycle
- L1d = 48 KiB, 12-way; Latency 4 cycles
- L2 = 1.25 MB; L3 = 24 MB (shared)

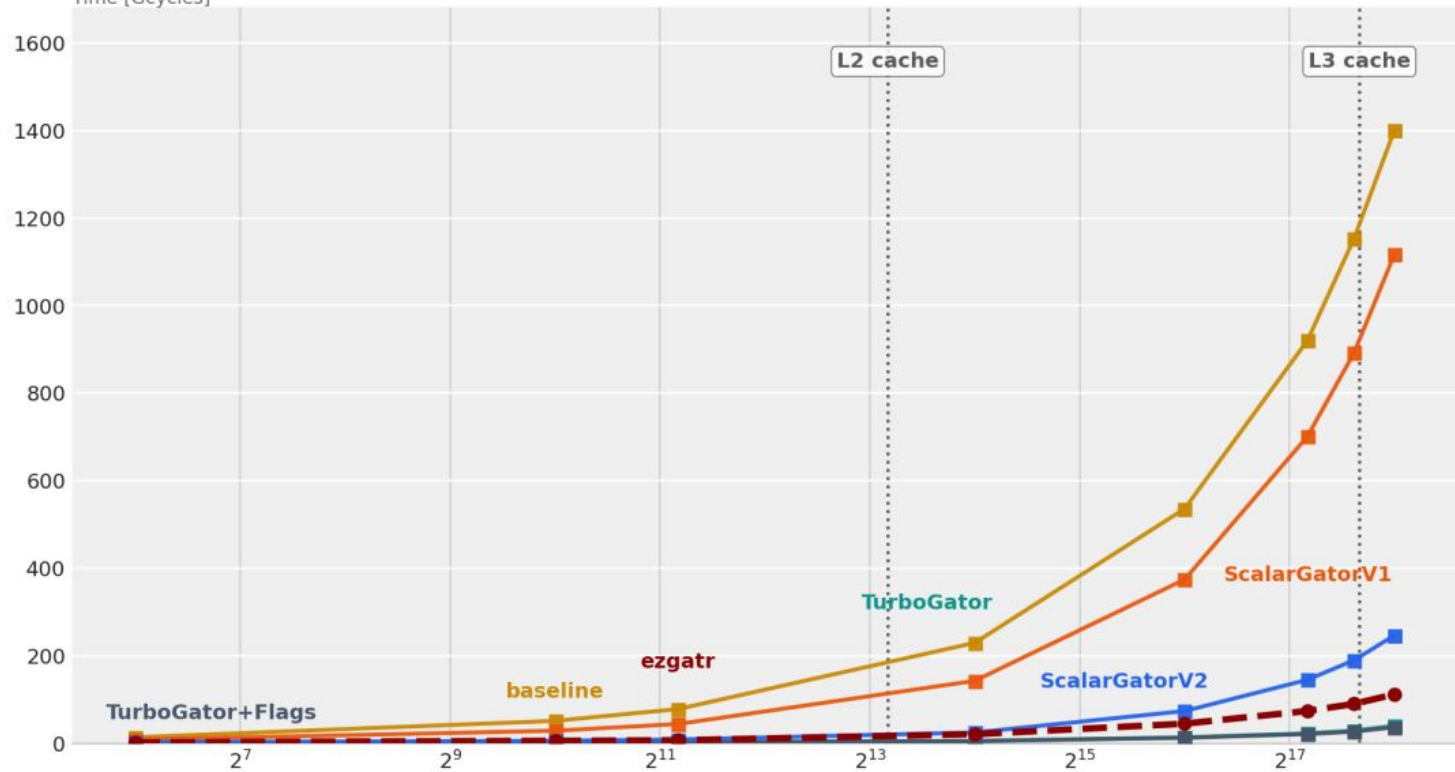
Performance: Intel Core 7 240H (P-core, 2.5GHz)

Performance [flops / cycle]



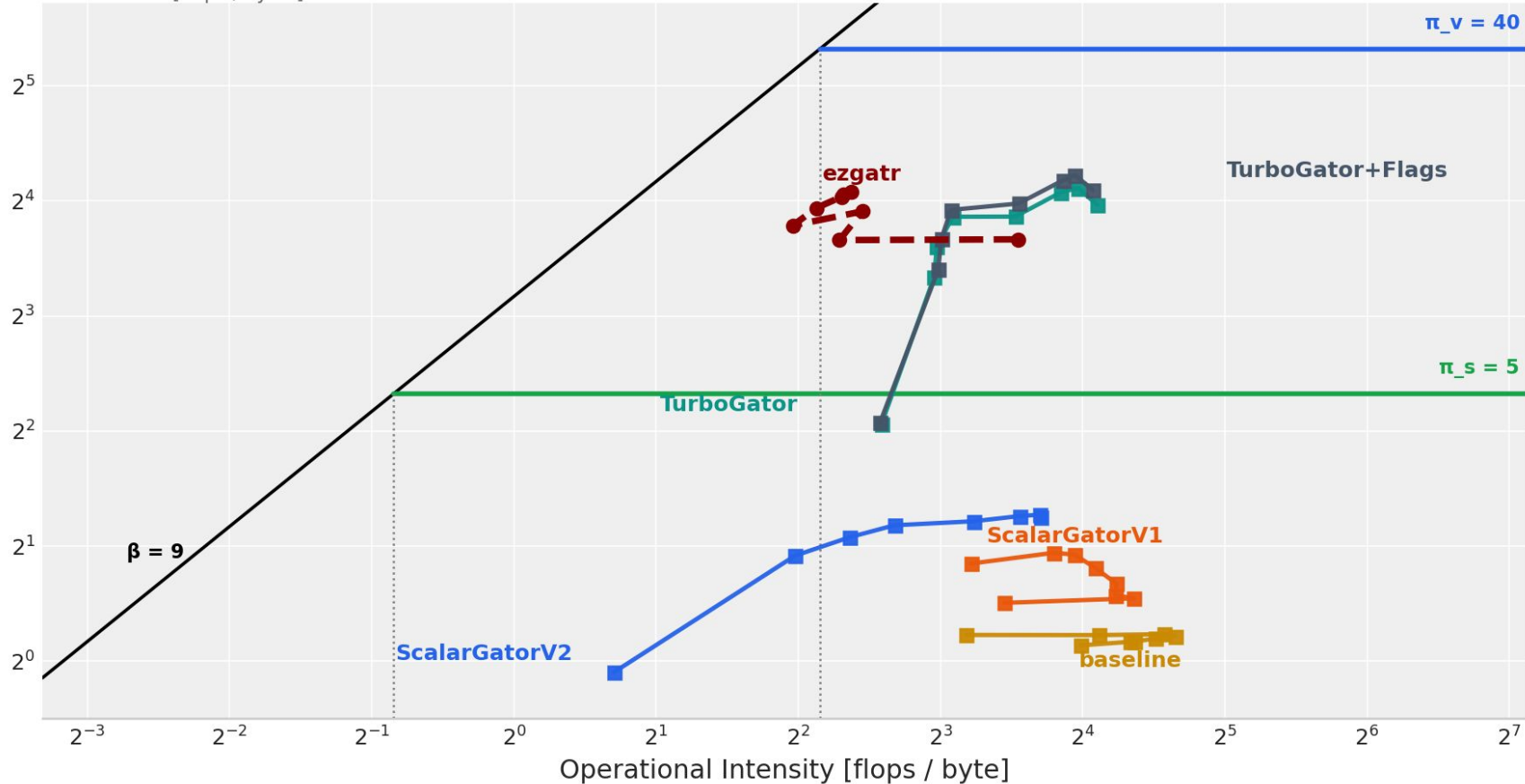
Runtime: Intel Core 7 240H (P-core, 2.5GHz) (With Baseline)

Time [Gcycles]

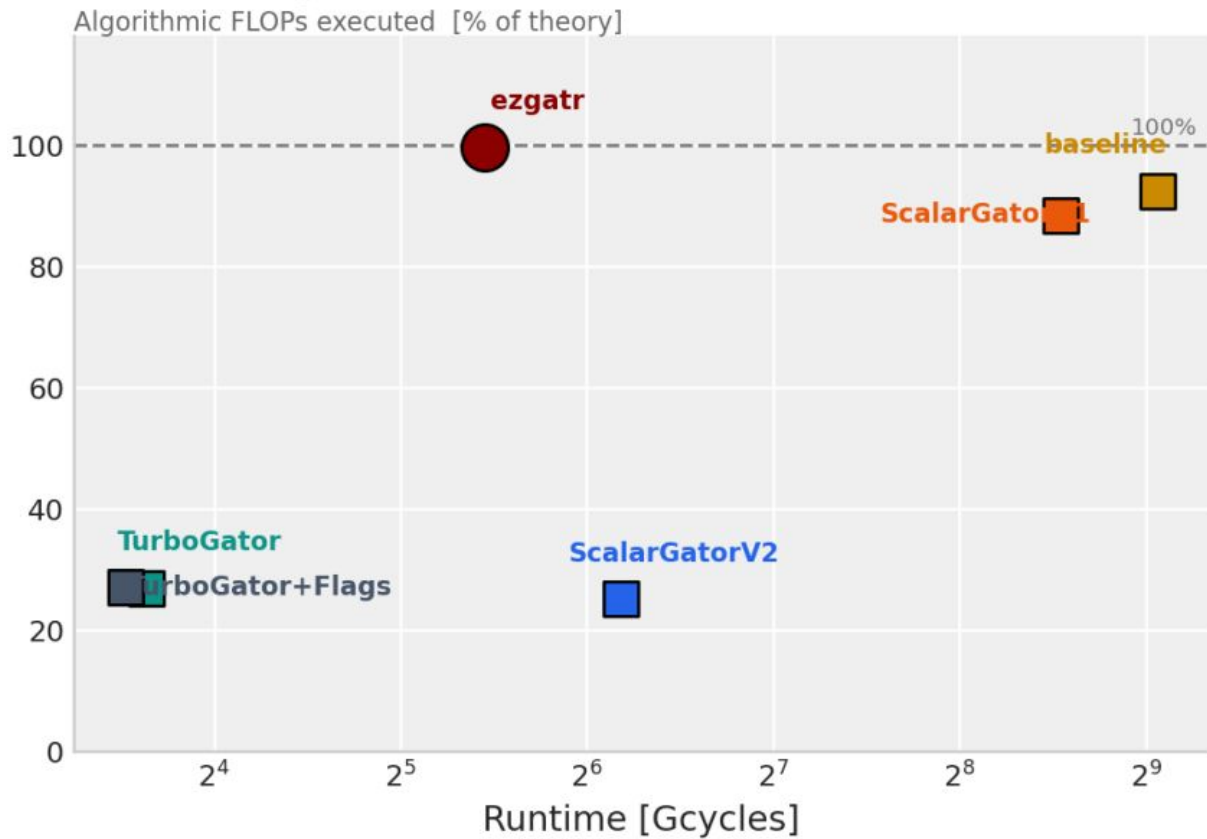


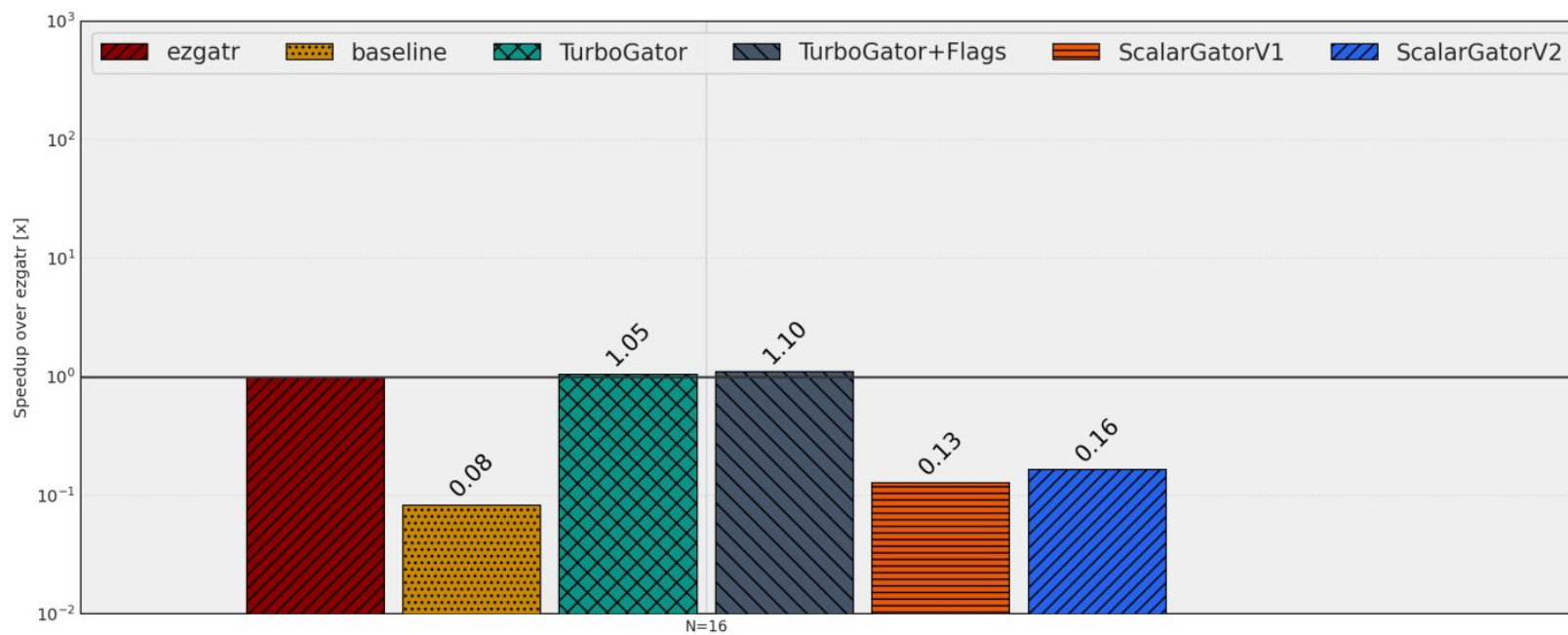
Roofline (DRAM): Intel Core 7 240H (P-core, 2.5GHz)

Performance [flops / cycle]



Work vs. Speed at N=32





The Bottleneck: Dense Math on Sparse Data

```
// BASELINE: General-Purpose Dense  
Execution  
for (int i = 0; i < 16; i++) {  
    for (int j = 0; j < 16; j++) {  
        for (int k = 0; k < 16; k++) {  
            // Wastes time multiplying by  
0.0 for 90% of the iterations  
            output[i] += basis[i][j][k] *  
x[j] * y[k];  
        }  
    }  
}
```

The Bottleneck: Dense Math on Sparse Data

```
// BASELINE: General-Purpose Dense  
Execution
```

```
for (int i = 0; i < 16; i++) {  
    for (int j = 0; j < 16; j++) {  
        for (int k = 0; k < 16; k++) {  
            // W  
0.0 for 90% of t  
            output[i] += basis[i][j][k]  
x[j] * y[k];  
        }  
    }  
}
```

```
def equi_linear(  
    Again, a map from "source-to-destination" style. Note that we may want  
    to optimize this operation as the basis is pretty sparse.
```

src/ezgatr/nn/functional/linear.py, line 250f.

The Bottleneck: Dense Math on Sparse Data

```
// BASELINE: General-Purpose Dense
Execution
for (int i = 0; i < 16; i++) {
    for (int j = 0; j < 16; j++) {
        for (int k = 0; k < 16; k++) {
            // Wastes time multiplying by
            0.0 for 90% of the iterations
            output[i] += basis[i][j][k] *
x[j] * y[k];
        }
    }
}
```

```
// OPTIMIZED: The Hardcoded "Recipe"
(Conceptual)
void geometric_product_kernel(float* x, float*
y, float* output) {
    // No zeros, just sign-flips

    output[0] = (x[0] * y[0]) + (x[1] * y[1]) +
(x[2] * y[2]);
    output[1] = (x[1] * y[0]) + (x[0] * y[1]) -
(x[3] * y[2]);
    output[2] = (x[2] * y[0]) + (x[0] * y[2]) +
(x[3] * y[1]);

    // ... continues for the 16 elements
}
```

Exploiting Sparsity: Hardcoding the Basis Product

```
// 80 iterations, mostly multiplying by zero
for (int i = 0; i < 4; ++i)
    for (int j = 0; j < 4; ++j)
        for (int k = 0; k < N_DAA; ++k)
            out[k] += basis[i][j][k] * r[i] * r[j];
```

Exploiting Sparsity: Hardcoding the Basis Product

```
// 80 iterations, mostly multiplying by zero
for (int i = 0; i < 4; ++i)
    for (int j = 0; j < 4; ++j)
        for (int k = 0; k < N_DAA; ++k)
            out[k] += basis[i][j][k] * r[i] * r[j];
```

↓
16x assignment
reduction 🙄

```
// 0 iterations, directly evaluating non-zeros
out[0] = -(r[3]*r[3]);
out[1] = -(r[0]*r[0] + r[1]*r[1] + r[2]*r[2]);
out[2] = 2.f * r[0]*r[3];
out[3] = 2.f * r[1]*r[3];
out[4] = 2.f * r[2]*r[3];
```

Exploiting Sparsity: Compact Basis Representation

```
// Allocates 4,096 elements (vast majority are zeros)
float data[16][16][16] = {};
LocalGPBasis()
{
    data[0][0][0] = 1.0f;
    data[0][2][2] = 1.0f;
    // ...
}
```

Exploiting Sparsity: Compact Basis Representation

```
// Allocates 4,096 elements (vast majority are zeros)
float data[16][16][16] = {};
LocalGPBasis()
{
    data[0][0][0] = 1.0f;
    data[0][2][2] = 1.0f;
    // ...
}
```

95% memory
footprint
reduction 🙄

```
// Allocates strictly the non-zero interactions
static constexpr GPEntry data[] = {
    {0,0,0,1}, {0,2,2,1}, {0,3,3,1}, {0,4,4,1},
    {0,8,8,-1}, {0,9,9,-1}, {0,10,10,-1}, {0,14,14,-1},
    // ... ~200 entries total
};
```


Optimizing Memory Access: Loop Permutation & Accumulation

```
for (int d = 0; d < 16; ++d) {  
    float sum = 0.0f;  
    for (int ic = 0; ic < in_channels; ++ic) {  
        for (int s = 0; s < 16; ++s) {  
            // 'x' is re-read from memory 16 times (once per 'd')  
            sum += kernel[...] * x[...] + ic * 16 + s];  
        }  
    }  
    out[...] = sum;  
}
```

Optimizing Memory Access: Loop Permutation & Accumulation

```
for (int d = 0; d < 16; ++d) {  
    float sum = 0.0f;  
    for (int ic = 0; ic < in_channels; ++ic) {  
        for (int s = 0; s < 16; ++s) {  
            // 'x' is re-read from memory 16 times (once per 'd')  
            sum += kernel[...] * x[...] + ic * 16 + s];  
        }  
    }  
    out[...] = sum;  
}
```



```
float acc[16] = {0.0f}; // Local register accumulator  
  
for (int ic = 0; ic < in_channels; ++ic) {  
    for (int s = 0; s < 16; ++s) {  
        // 'x' is loaded exactly ONCE  
        float xv = x[...] + ic * 16 + s];  
        for (int d = 0; d < 16; ++d) {  
            acc[d] += kernel[...] * xv;  
        }  
    }  
}  
// ... write acc[d] array to main memory
```

16x memory
reads for x
reduction 🤔

From Fragmented to Flattened: Contiguous Memory Access

```
// BEFORE: Fragmented Memory & Redundant Math
for (int t1 = 0; t1 < T; ++t1) {
    for (int t2 = 0; t2 < T; ++t2) {
        for (int c = 0; c < C; ++c) {
            // Scattered Memory Jumps (Cache Misses)
            float* mv_q = q_base + t1 * stride + c * 16;
            float* mv_k = k_base + t2 * stride + c * 16;
            // Redundant Math (Calculated T*T times)
            project_features(mv_q, pq);
            project_features(mv_k, pk);
            // Actual Dot Product
            for (int j = 0; j < FEATURES; ++j) {
                s += pq[j] * pk[j];
            }
        }
    }
}
```

From Fragmented to Flattened: Contiguous Memory Access

```
// BEFORE: Fragmented Memory & Redundant Math
for (int t1 = 0; t1 < T; ++t1) {
    for (int t2 = 0; t2 < T; ++t2) {
        for (int c = 0; c < C; ++c) {
            // Scattered Memory Jumps (Cache Misses)
            float* mv_q = q_base + t1 * stride + c * 16;
            float* mv_k = k_base + t2 * stride + c * 16;
            // Redundant Math (Calculated T*T times)
            project_features(mv_q, pq);
            project_features(mv_k, pk);
            // Actual Dot Product
```

```
// AFTER: Flattened Memory & Contiguous Access
// (Features are already projected & packed sequentially)
for (int t1 = 0; t1 < T; ++t1) {
    float* qf = q_flat + t1 * qk_dim;
    for (int t2 = 0; t2 < T; ++t2) {
        float* kf = k_flat + t2 * qk_dim;
        // Straight-line, contiguous memory reads (Cache Hit!)
        // No redundant math inside the loop
        for (int d = 0; d < qk_dim; ++d) {
            s += qf[d] * kf[d];
        }
    }
}
```

From Cache Thrashing to Cache Blocking

```
// BEFORE: Unblocked Execution (Spills out of CPU Cache)
for (int t1 = 0; t1 < T; ++t1) {
    for (int t2 = 0; t2 < T; ++t2) {
        float s = 0.f;

        // Reads the ENTIRE sequence at once, pushing old data out of cache
        for (int d = 0; d < qk_dim; ++d) {
            s += qf1[d] * kf2[d];
        }

        scores[t1 * T + t2] = s;
    }
}
```

From Cache Thrashing to Cache Blocking

```
// BEFORE: Unblocked Execution (Spills out of CPU Cache)
for (int t1 = 0; t1 < T; ++t1) {
    for (int t2 = 0; t2 < T; ++t2) {

// AFTER: Blocked Execution (Locks data in L1 Cache)
// BT = Block size for Tokens, BD = Block size for Dimensions
// "Managers": Fetch chunks of data that fit perfectly in cache
for (int t1b = 0; t1b < T; t1b += BT) {
    for (int t2b = 0; t2b < T; t2b += BT) {
        for (int db = 0; db < qk_dim; db += BD) {
            // "Workers": Do math STRICTLY within the current block boundaries
            for (int t1 = t1b; t1 < t1b + BT; ++t1) {
                for (int t2 = t2b; t2 < t2b + BT; ++t2) {
                    float s = 0.f;
                    for (int d = db; d < db + BD; ++d) {
                        s += qf1[d] * kf2[d];
                    }
                    // Accumulate the partial sum into the final score
                    scores[t1 * T + t2] += s;
                }
            }
        }
    }
}
}
```

Profiling

Server/Compiler Setup

Performance Plots

Server Machine

Validation against ezgatr

All our versions are validated against ezgatr on submission

acceptable thresholds:

- **0.01% relative error**
- **0.00001 absolute error**

(in reality much lower)

Benchmarking sweeps

SIZES = [1, 4, 6, 16, 32, 48, 56, 64]

X = (B, T, C_in, D) = (8, 32n, 2n, 16)

	target	obs worst
for each job	max/min < 1.10x	1.009x
warmup(5)	iqr < 0.5% median	0.14%
measure(9)	std < 1% mean	0.27%
observed	CV% < 1%	0.27%

exception: Baseline w=3 , m=5 [runtime too high], still 0.06 CV%

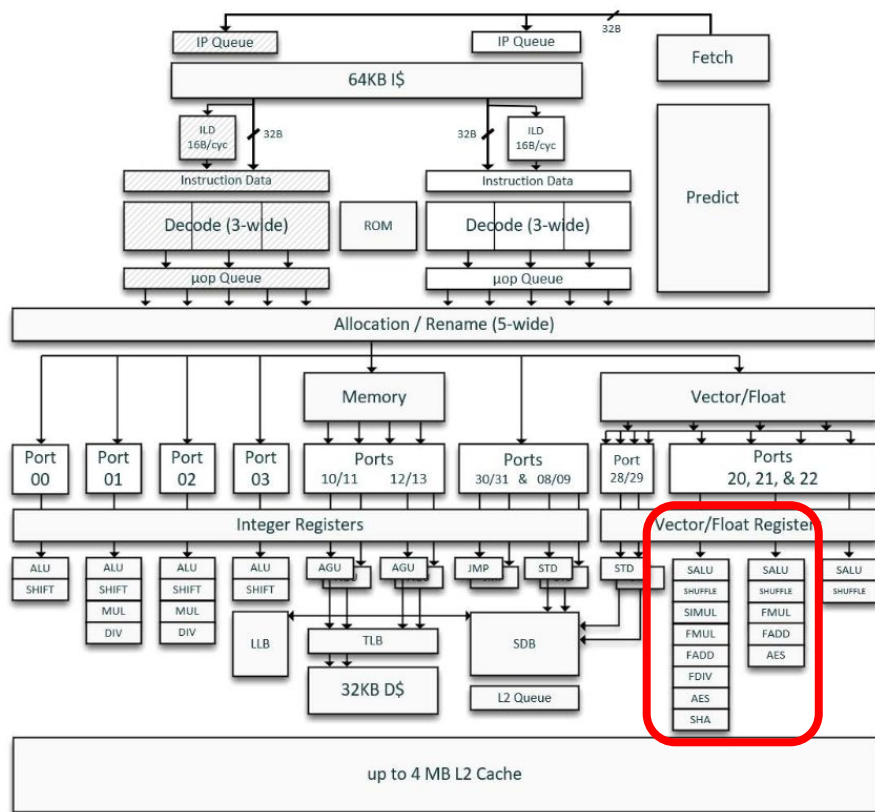
Left to do

Interested in trying to change the python "glue" of how the functions are chained to maybe merge things and break barriers.

like `geom_product + equi_join` = a single method keeping things in registers?

theoretical microbenchmarking individual functions

Recap Server setup



Intel N100 Alder Lake E core (Gracemont)

Intel SSE4.1, Intel SSE4.2, Intel AVX2, FMA3

TurboBoost off 800MHz, 12 GB RAM

256b instructions get "cracked" in two 128b instruction

$\pi_s = 3.33$ flops/cycle

$\pi_v = 16$ flops/cycle

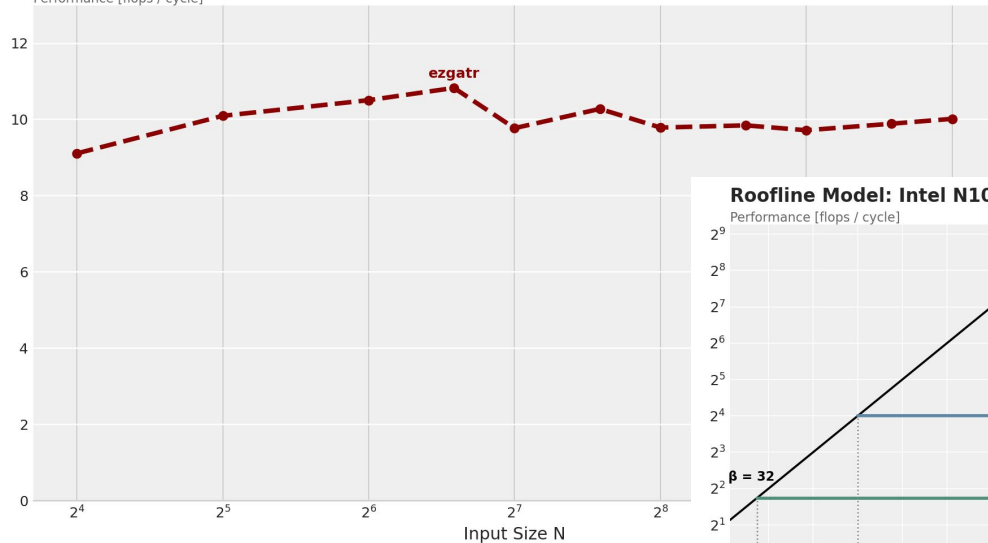
$\beta = 32$ bytes/cycle

L1d = 32 KiB; Latency 3 Cycles

Last time on TurboGator...

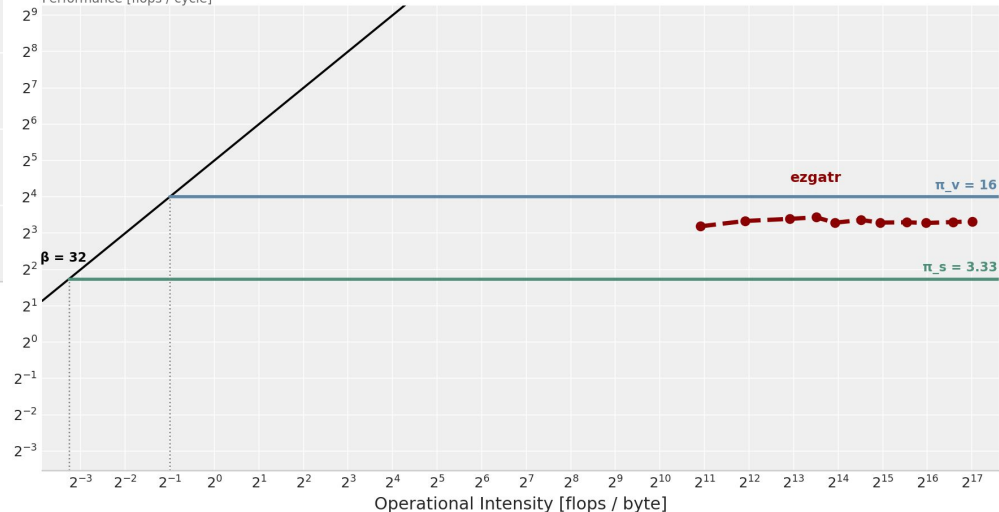
Performance: Intel N100 (1 Core, 800Mhz)

Performance [flops / cycle]



Roofline Model: Intel N100 (1 Core, 800Mhz)

Performance [flops / cycle]



New sweep

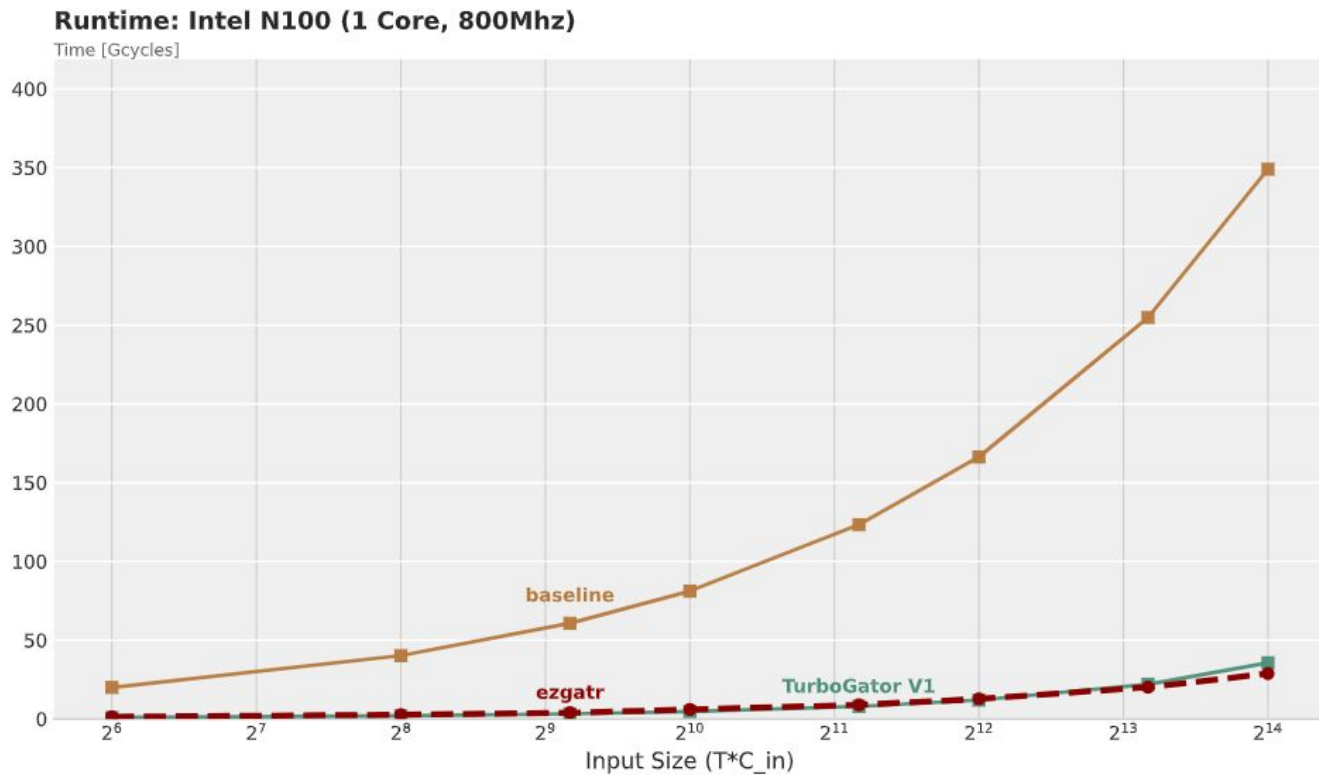
$$X = (8, T, C_{in}, 16)$$

$$T = 32 * N$$

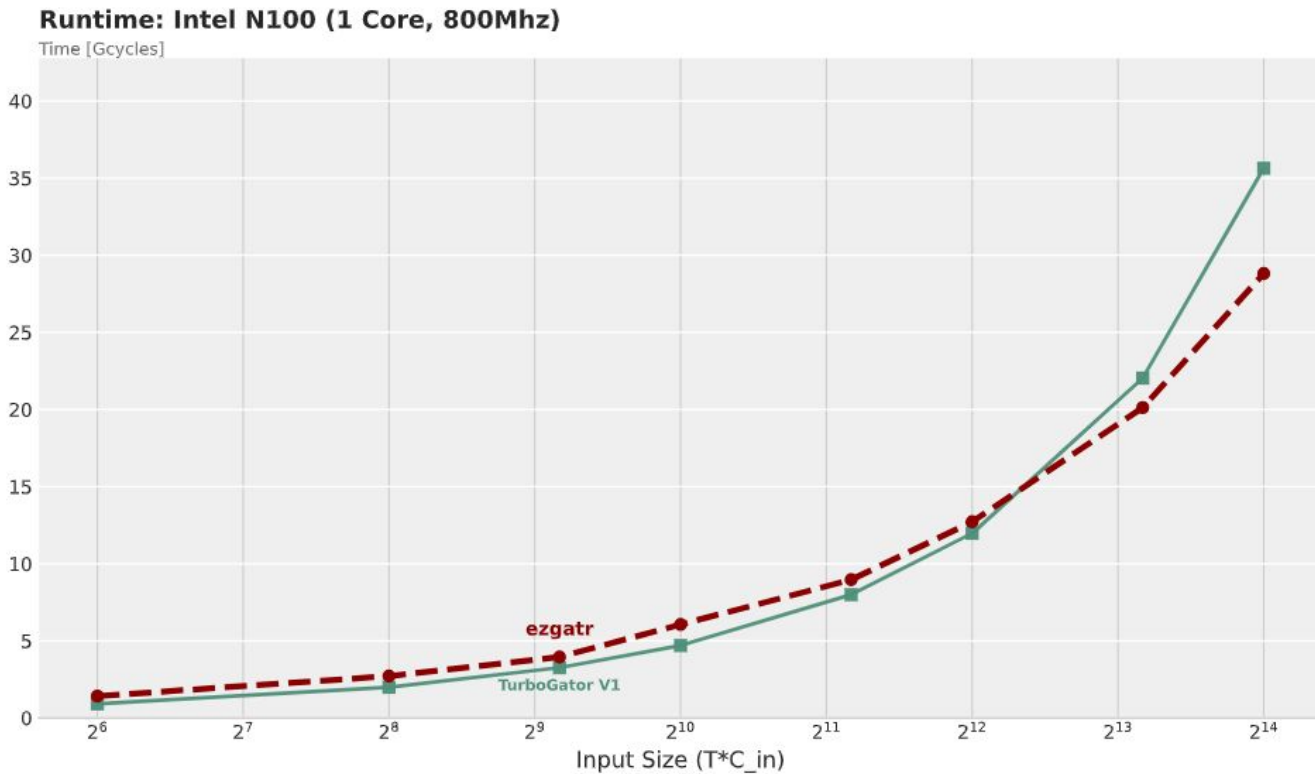
$$C_{in} = 2 * N$$

N=1 -> (32, 2)	= 64
N=2 -> (64, 4)	= 256
N=3 -> (96, 6)	= 576
N=4 -> (128, 8)	= 1024
N=6 -> (192, 12)	= 2304
N=8 -> (256, 16)	= 4096
N=12 -> (384, 24)	= 9216
N=16 -> (512, 32)	= 16384

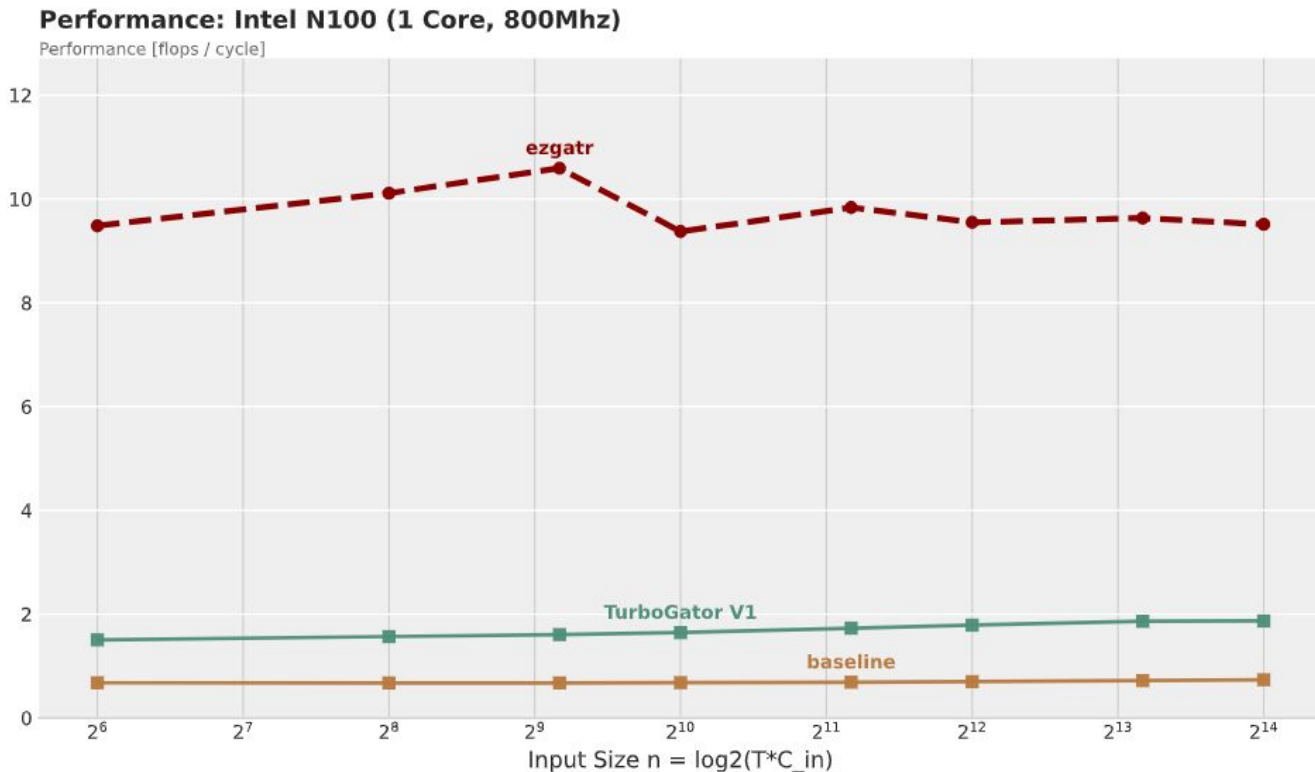
Runtime Today



Runtime Today (ignore baseline)



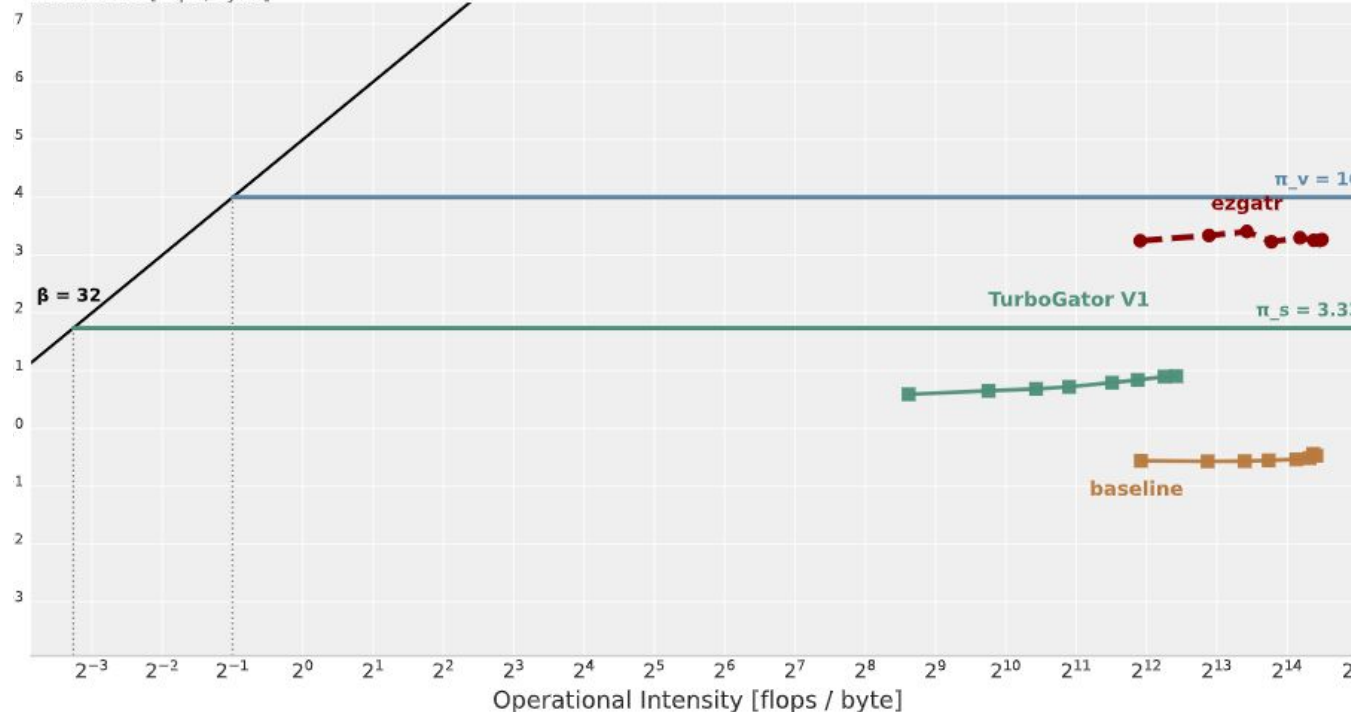
Performance Today



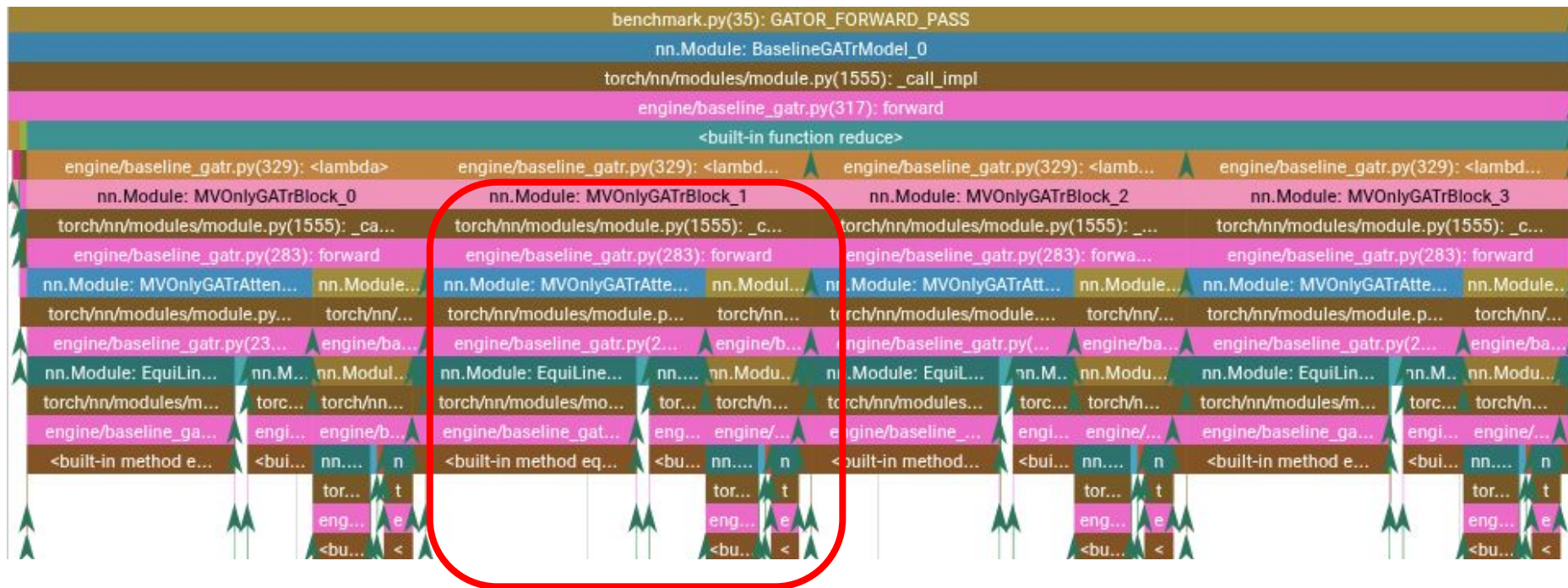
Roofline Today

Roofline Model: Intel N100 (1 Core, 800Mhz)

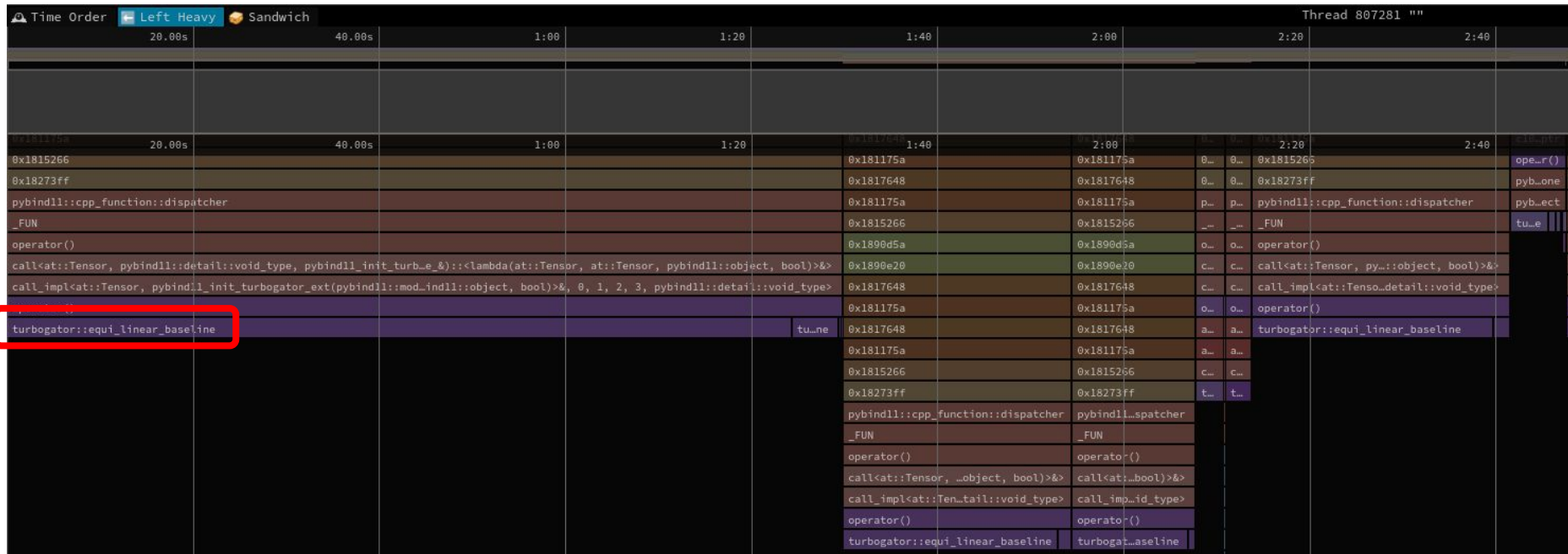
Performance [flops / cycle]



Profiling (pytorch Profiler)



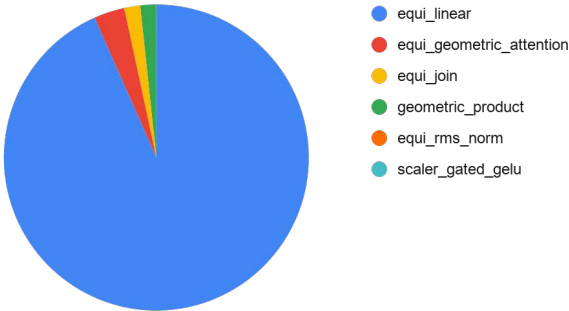
Profiling (py-spy)



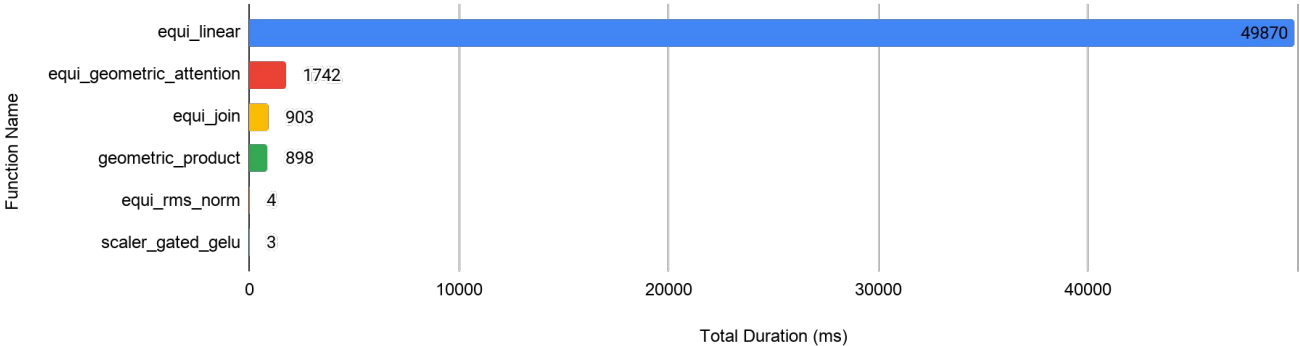
Bottlenecks identified

Function Name	Call Count	Total Duration (ms)	% of Total Trace
equi_linear	22	49870	92
equi_geometric_attention	4	1742	3
equi_join	4	903	1
geometric_product	4	898	1
equi_rms_norm	8	4	0.007
scaler_gated_gelu	4	3	0.006

Distribution of Trace Time by Function



Total Duration by Baseline Function



Bottleneck Optimizations

- Simplicity
- Scalar Replacement
- Sparsity
- Memory Layout Optimizations
 - Flattening & Contiguous vectors
 - Blocking
- Vectorization with AVX (coming soon)

Simplicity

```
for (int i = 0; i < 4; ++i)
    for (int j = 0; j < 4; ++j)
        for (int k = 0; k < N_DAA; ++k)
            out[k] += basis[i][j][k] * r[i] * r[j];
```



```
out[0] = -(r[3]*r[3]);
out[1] = -(r[0]*r[0] + r[1]*r[1] + r[2]*r[2]);
out[2] = 2.f * r[0]*r[3];
out[3] = 2.f * r[1]*r[3];
out[4] = 2.f * r[2]*r[3];
```

Scalar replacements

```
// step 2: out[b,o,d] = sum_{i,s} kernel[o,i,d,s] * x[b,i,s]
for (size_t b = 0; b < batch; ++b)
{
    for (size_t oc = 0; oc < out_channels; ++oc)
    {
        for (size_t d = 0; d < 16; ++d)
        {
            float sum = 0.0f;
            for (size_t ic = 0; ic < in_channels; ++ic)
            {
                for (size_t s = 0; s < 16; ++s)
                {
                    sum += kernel[((oc * in_channels + ic) * 16 + d) * 16 + s]
                        * x[b * in_channels * 16 + ic * 16 + s];
                }
            }
            out[b * out_channels * 16 + oc * 16 + d] = sum;
        }
    }
}
```



```
// step 2: out[b,o,d] = sum_{i,s} kernel[o,ic,s,d] * x[b,ic,s]
// OPT1:
// - register accumulator: acc[16] holds all d values for one (b, oc)
// - x[b,ic,s] is loaded once per (ic,s) and reused across all 16 d
for (size_t b = 0; b < batch; ++b)
{
    for (size_t oc = 0; oc < out_channels; ++oc)
    {
        float acc[16] = {0.0f};
        for (size_t ic = 0; ic < in_channels; ++ic)
        {
            for (size_t s = 0; s < 16; ++s)
            {
                float xv = x[b * in_channels * 16 + ic * 16 + s];
                for (size_t d = 0; d < 16; ++d)
                {
                    acc[d] += kernel[((oc * in_channels + ic) * 16 + s) * 16 + d] * xv;
                }
            }
        }
        for (size_t d = 0; d < 16; ++d)
        {
            out[b * out_channels * 16 + oc * 16 + d] = acc[d];
        }
    }
}
```

Sparsity

```
float data[16][16][16] = {};  
LocalGPBasis()  
{  
    data[0][0][0] = 1.0f;  
    data[0][2][2] = 1.0f;  
    data[0][3][3] = 1.0f;  
    data[0][4][4] = 1.0f;  
    data[0][8][8] = -1.0f;  
    data[0][9][9] = -1.0f;  
    data[0][10][10] = -1.0f;  
    data[0][14][14] = -1.0f;  
    data[1][0][1] = 1.0f;  
    ...  
}
```

95% reduction 🤨

```
static constexpr GPEntry data[] = {  
    {0, 0, 0, 1}, {0, 2, 2, 1}, {0, 3, 3, 1}, {0, 4, 4, 1},  
    {0, 8, 8, -1}, {0, 9, 9, -1}, {0, 10, 10, -1}, {0, 14, 14, -1},  
    {1, 0, 1, 1}, {1, 1, 0, 1}, {1, 2, 5, -1}, {1, 3, 6, -1},  
    {1, 4, 7, -1}, {1, 5, 2, 1}, {1, 6, 3, 1}, {1, 7, 4, 1},  
    {1, 8, 11, -1}, {1, 9, 12, -1}, {1, 10, 13, -1}, {1, 11, 8, -1},  
    {1, 12, 0, -1}, {1, 13, 10, -1}, {1, 14, 15, 1}, {1, 15, 14, -1},  
    {2, 0, 2, 1}, {2, 2, 0, 1}, {2, 3, 8, -1}, {2, 4, 9, -1},  
    {2, 8, 3, 1}, {2, 9, 4, 1}, {2, 10, 14, -1}, {2, 14, 10, -1},  
    {3, 0, 3, 1}, {3, 2, 8, 1}, {3, 3, 0, 1}, {3, 4, 10, -1},  
    {3, 8, 2, -1}, {3, 9, 14, 1}, {3, 10, 4, 1}, {3, 14, 9, 1},  
    {4, 0, 4, 1}, {4, 2, 9, 1}, {4, 3, 10, 1}, {4, 4, 0, 1},  
    {4, 8, 14, -1}, {4, 9, 2, -1}, {4, 10, 3, -1}, {4, 14, 8, -1},  
    {5, 0, 5, 1}, {5, 1, 2, 1}, {5, 2, 1, -1}, {5, 3, 11, 1},  
    {5, 4, 12, 1}, {5, 5, 0, 1}, {5, 6, 8, -1}, {5, 7, 9, -1},  
    {5, 8, 6, 1}, {5, 9, 7, 1}, {5, 10, 15, -1}, {5, 11, 3, 1},  
    {5, 12, 4, 1}, {5, 13, 14, -1}, {5, 14, 13, 1}, {5, 15, 10, -1},  
    {6, 0, 6, 1}, {6, 1, 3, 1}, {6, 2, 11, -1}, {6, 3, 1, -1},  
    {6, 4, 13, 1}, {6, 5, 8, 1}, {6, 6, 0, 1}, {6, 7, 10, -1},  
    {6, 8, 5, -1}, {6, 9, 15, 1}, {6, 10, 7, 1}, {6, 11, 2, -1},  
    {6, 12, 14, 1}, {6, 13, 4, 1}, {6, 14, 12, -1}, {6, 15, 9, 1},  
    {7, 0, 7, 1}, {7, 1, 4, 1}, {7, 2, 12, -1}, {7, 3, 13, -1},  
    {7, 4, 1, -1}, {7, 5, 9, 1}, {7, 6, 10, 1}, {7, 7, 0, 1},  
    {7, 8, 15, -1}, {7, 9, 5, -1}, {7, 10, 6, -1}, {7, 11, 14, -1},  
    {7, 12, 2, -1}, {7, 13, 3, -1}, {7, 14, 11, 1}, {7, 15, 8, -1},  
    {8, 0, 8, 1}, {8, 2, 3, 1}, {8, 3, 2, -1}, {8, 4, 14, 1},  
    {8, 8, 0, 1}, {8, 9, 10, -1}, {8, 10, 9, 1}, {8, 14, 4, 1},  
    {9, 0, 9, 1}, {9, 2, 4, 1}, {9, 3, 14, -1}, {9, 4, 2, -1},  
    {9, 8, 10, 1}, {9, 9, 0, 1}, {9, 10, 8, -1}, {9, 14, 3, -1},  
    {10, 0, 10, 1}, {10, 2, 14, 1}, {10, 3, 4, 1}, {10, 4, 3, -1},  
    {10, 8, 9, -1}, {10, 9, 8, 1}, {10, 10, 0, 1}, {10, 14, 2, 1},  
    {11, 0, 11, 1}, {11, 1, 8, 1}, {11, 2, 6, -1}, {11, 3, 5, 1},  
    {11, 4, 15, -1}, {11, 5, 3, 1}, {11, 6, 2, -1}, {11, 7, 14, 1},  
    {11, 8, 1, 1}, {11, 9, 13, -1}, {11, 10, 12, 1}, {11, 11, 0, 1},  
    {11, 12, 10, -1}, {11, 13, 9, 1}, {11, 14, 7, -1}, {11, 15, 4, 1},  
    {12, 0, 12, 1}, {12, 1, 9, 1}, {12, 2, 7, -1}, {12, 3, 15, 1},  
    {12, 4, 5, 1}, {12, 5, 4, 1}, {12, 6, 14, -1}, {12, 7, 2, -1},  
    {12, 8, 13, 1}, {12, 9, 1, 1}, {12, 10, 11, -1}, {12, 11, 10, 1},  
    {12, 12, 0, 1}, {12, 13, 8, -1}, {12, 14, 6, 1}, {12, 15, 3, -1},  
    {13, 0, 13, 1}, {13, 1, 10, 1}, {13, 2, 15, -1}, {13, 3, 7, -1},  
    {13, 4, 6, 1}, {13, 5, 14, 1}, {13, 6, 4, 1}, {13, 7, 3, -1},  
    {13, 8, 12, -1}, {13, 9, 11, 1}, {13, 10, 1, 1}, {13, 11, 9, -1},  
    {13, 12, 8, 1}, {13, 13, 0, 1}, {13, 14, 5, -1}, {13, 15, 2, 1},  
    {14, 0, 14, 1}, {14, 2, 10, 1}, {14, 3, 9, -1}, {14, 4, 8, 1},  
    {14, 8, 4, 1}, {14, 9, 3, -1}, {14, 10, 2, 1}, {14, 14, 0, 1},  
    {15, 0, 15, 1}, {15, 1, 14, 1}, {15, 2, 13, -1}, {15, 3, 12, 1},  
    {15, 4, 11, -1}, {15, 5, 10, 1}, {15, 6, 9, -1}, {15, 7, 8, 1},  
    {15, 8, 7, 1}, {15, 9, 6, -1}, {15, 10, 5, 1}, {15, 11, 4, 1},  
    {15, 12, 3, -1}, {15, 13, 2, 1}, {15, 14, 1, -1}, {15, 15, 0, 1},  
};
```

Sparsity II

```
float unnorm_basis[9][16][16] = {};  
float norm_basis[9][16][16] = {};  
  
LocalEquiLinearBasis()  
{  
    const std::vector<std::vector<BasisElement>> basis_elements = {  
        {0},  
        {1, 2, 3, 4},  
        {5, 6, 7, 8, 9, 10},  
        {11, 12, 13, 14},  
        {15},  
        {std::pair<int, int>{1, 0}},  
        {std::pair<int, int>{5, 2}, std::pair<int, int>{6, 3}, std::pair<int, int>{7, 4}},  
        {std::pair<int, int>{11, 8}, std::pair<int, int>{12, 9}, std::pair<int, int>{13, 10}},  
        {std::pair<int, int>{15, 14}},  
    };  
};
```

Sparsity II

```
for (size_t b = 0; b < batch; ++b)
{
    for (size_t oc = 0; oc < out_channels; ++oc)
    {
        float acc[16] = {0.0f};

        for (size_t ic = 0; ic < in_channels; ++ic)
        {
            for (size_t s = 0; s < 16; ++s)
            {
                float xv = x[b * in_channels * 16 + ic * 16 + s];
                for (size_t d = 0; d < 16; ++d)
                {
                    acc[d] += kernel[(oc * in_channels + ic) * 16 + s] * xv;
                }
            }

            for (size_t d = 0; d < 16; ++d)
            {
                out[b * out_channels * 16 + oc * 16 + d] = acc[d];
            }
        }
    }
}
```

256 $\rightarrow$ 26 FMAs per
(b, oc, ic)



```
for (size_t ic = 0; ic < in_channels; ++ic)
{
    const float *w_oi = w_use.data() + (oc * in_channels + ic) * 9;
    const float *x_bi = x + (b * in_channels + ic) * 16;

    // load weights for (oc, ic)
    const float w0 = w_oi[0];
    const float w1 = w_oi[1];
    const float w2 = w_oi[2];
    const float w3 = w_oi[3];
    const float w4 = w_oi[4];
    const float w5 = w_oi[5];
    const float w6 = w_oi[6];
    const float w7 = w_oi[7];
    const float w8 = w_oi[8];

    // w=0
    acc[0] += w0 * x_bi[0];

    // w=1
    acc[1] += w1 * x_bi[1];
    acc[2] += w1 * x_bi[2];
    acc[3] += w1 * x_bi[3];
    acc[4] += w1 * x_bi[4];

    // w=2
    acc[5] += w2 * x_bi[5];
    acc[6] += w2 * x_bi[6];
    acc[7] += w2 * x_bi[7];
    acc[8] += w2 * x_bi[8];
    acc[9] += w2 * x_bi[9];
    acc[10] += w2 * x_bi[10];
}
```

...

Memory Layout - Flatting & Cont. Vec

- Flattened Q, K vector projections and ensured contiguous memory access.
- Together with blocking significantly improves cache efficiency for GEMM score computation.

q:	7 IPA Blades	5 DAA Blades
-----------	---------------------	---------------------

Memory Layout - Blocking

- Currently only implemented in `equi_geom_attn`
 - Around 2x runtime improvement, more efficient caching
- Coming for `equi_linear` in V1.1
 - We expect massive gains, since the runtime share of `equi_linear` is much larger than `equi_geom_attn`

Future considerations

- Vectorisation
- Automatic FLOPs and Data Movement
- Intel VTune didn't work, perf flags missing (counters missing from silicon)
- We have to change server infra...